CHAPTER 7

Bitcoin Clients and APIs

Installation, Configuration, Networks & Programming

TC789 – Teknologi Blockchain

Teguh Indra Bayu, Ph.D. | FTI UKSW

Source: Mastering Blockchain, 2nd Edition — I. Bashir, Packt Publishing

Learning Objectives

1. Identify the three types of Bitcoin Core clients and their roles
2. Install and configure a Bitcoin node on a Linux system
3. Describe the purpose and location of the bitcoin.conf configuration file
4. Distinguish between Mainnet, Testnet, and Regtest network modes
5. Use bitcoin-cli commands to interact with the Bitcoin RPC interface
6. List available programming libraries and APIs for Bitcoin development
7. Apply security evaluation criteria when choosing a Bitcoin API

Chapter Overview

What this chapter covers

01

Bitcoin Core Installation

Download, install, and build Bitcoin Core from source on multiple platforms

02

Client Types & Configuration

bitcoind, bitcoin-cli, bitcoin-qt — roles, differences, and bitcoin.conf setup

03

Network Modes

Mainnet, Testnet, and Regtest — choosing the right environment for your work

04

CLI Commands

Using bitcoin-cli with JSON-RPC to query the blockchain and control the node

05

Bitcoin APIs

Online APIs: Blockchain.info, BitPay, Block.io, Bitcore, BitcoinJS

06

Programming Libraries

Libbitcoin (C++), Pycoin (Python), Bitcoinj (Java)

Installing Bitcoin Core

Downloading the reference implementation

Official Download
https://bitcoin.org/en/download

Supported Platforms

- Windows x86 / x64
- Linux x86 / x64 / ARM
- macOS

Open-Source Repository
github.com/bitcoin/bitcoin

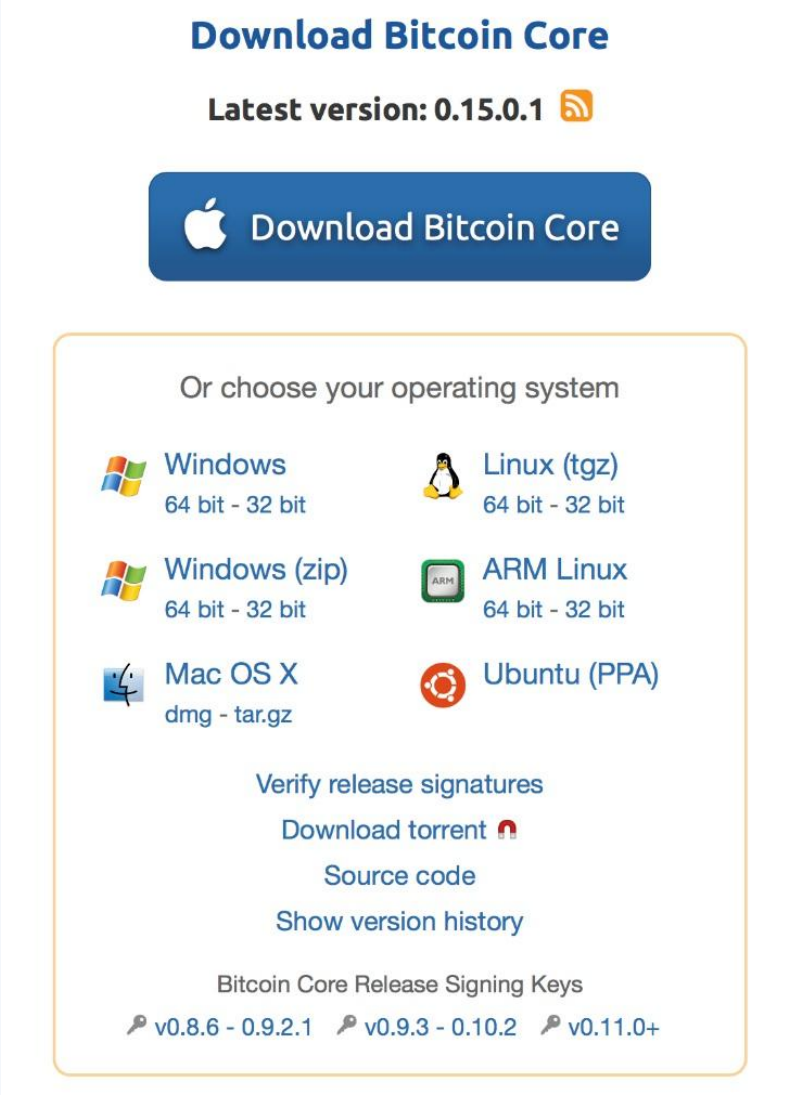


Figure: Bitcoin Core download page (bitcoin.org)

Three Types of Bitcoin Core Clients

bitcoind

Daemon (Background Process)

- Core software running as a system daemon
- Maintains the full blockchain copy
- Exposes the JSON-RPC interface to other clients
- Does NOT have a user interface

bitcoin-cli

Command-Line Interface (CLI)

- Feature-rich CLI tool for interacting with bitcoind
- Calls JSON-RPC functions on the daemon
- Does NOT independently act on the blockchain
- Used for scripting and automation

bitcoin-qt

Graphical User Interface (GUI)

- Full-featured GUI wallet application
- First verifies blocks on disk, then displays GUI
- Block verification is shared with bitcoind
- Suitable for non-technical end users

Bitcoin-qt: The GUI Client

Graphical interface for Bitcoin Core

Key Behaviors

First Launch

Verifies all blocks stored on disk before the GUI appears

Shared Verification

Block verification is not specific to bitcoin-qt; bitcoind performs the same process

Full Node

Downloads and validates the entire Bitcoin blockchain

Wallet Included

Manages keys, addresses, and transactions via GUI

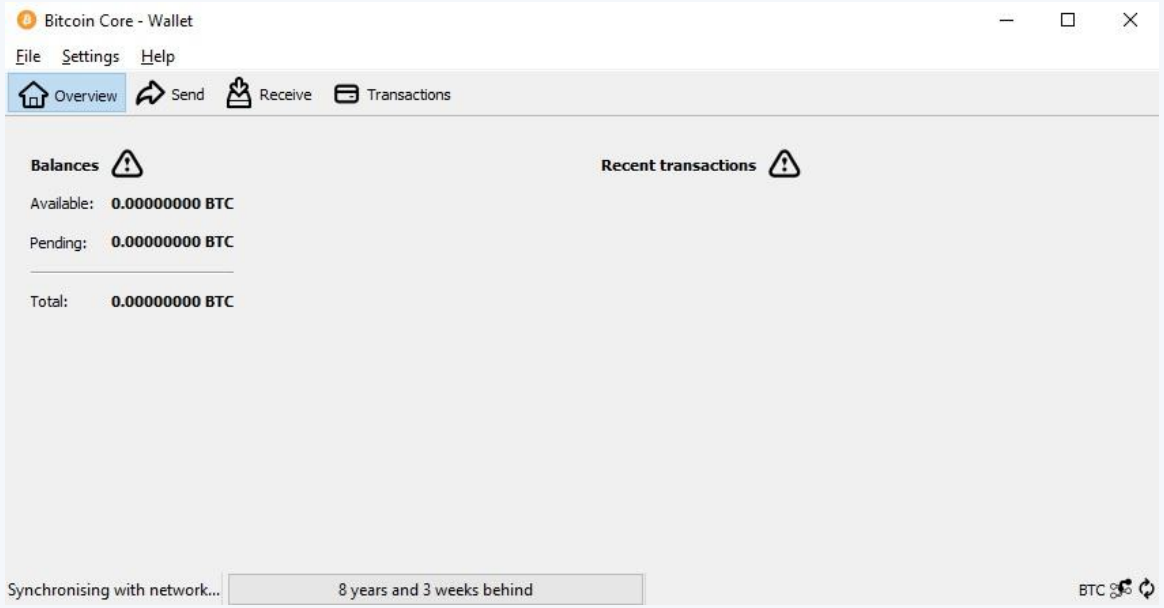


Figure: Bitcoin Core QT client after installation — blockchain not yet synced

Setting Up a Bitcoin Node

Ubuntu/Linux installation walkthrough

Step-by-Step Install

Step 1 — *Update package lists*

```
sudo apt-get update
```

Step 2 — *Install daemon client*

```
sudo apt-get install bitcoind
```

Step 3 — *Install GUI client*

```
sudo apt-get install bitcoin-qt
```

Step 4 — *Install both clients at once*

```
sudo apt-get install bitcoin-qt bitcoind
```

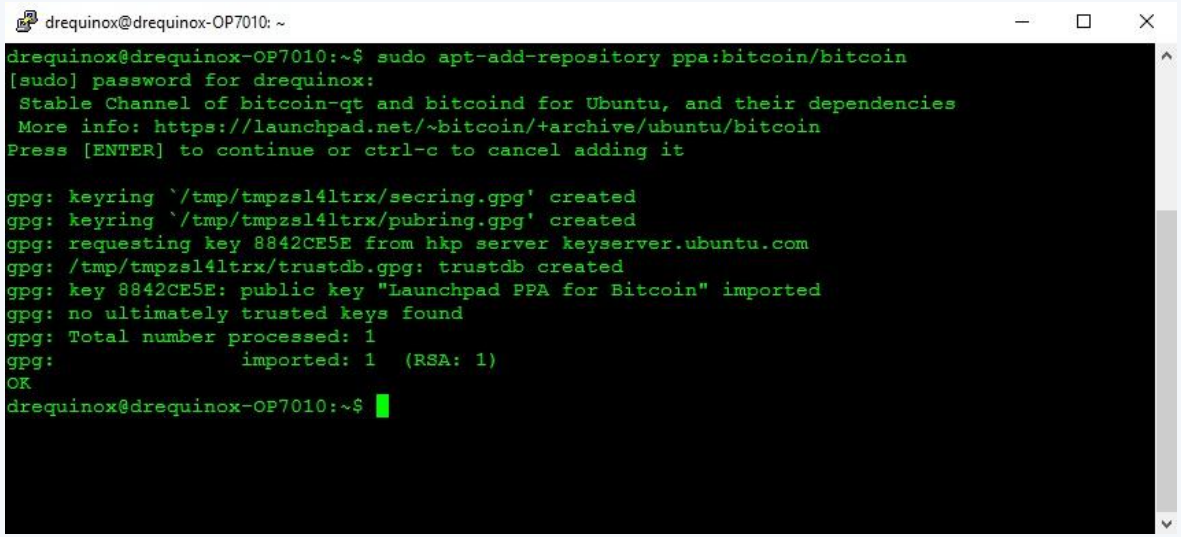


Figure: Bitcoin Core setup process on Ubuntu

Building from Source Code

SOURCE BUILD

For developers who want to contribute or learn

Build Process

Install git

```
$ sudo apt-get install git
```

Create directory

```
$ mkdir bcsource && cd bcsource
```

Clone repository

```
$ git clone https://github.com/bitcoin/bitcoin.git
```

Change directory

```
$ cd bitcoin
```

Generate build files

```
$ ./autogen.sh
```

Configure build

```
$ ./configure.sh
```

Compile source

```
$ make
```

Install binaries

```
$ sudo make install
```

Why build from source? Contribute to Bitcoin development, inspect the code, or customize behavior.

bitcoin.conf Configuration File

CONFIGURATION

Centralizing runtime settings

bitcoin.conf is read by both bitcoind and bitcoin-qt at startup. It eliminates the need to type long command-line arguments every time.

Default Location (Linux)

`$HOME/.bitcoin/bitcoin.conf`

Exception

The -conf switch itself CANNOT be set inside the file

Custom Location

`bitcoind -conf=<filepath>`

Auto-loaded

Read automatically at every startup — no manual import needed

Scope

Applies to bitcoind and bitcoin-qt

Precedence

Command-line flags override bitcoin.conf values

Bitcoin Network Modes

NETWORK MODES

Choosing the right environment

LIVE

Mainnet (Live Network)

- Real Bitcoin with real monetary value
- Standard mining difficulty (10 min blocks)
- Full global peer-to-peer network
- Irreversible transactions

Production use only

TEST

Testnet (Public Test)

- Test bitcoins — no real value
- Relaxed mining rules, faster blocks
- Public peer-to-peer test network
- Free coins via faucet services

Testing before deployment

DEV

Regtest (Local Testing)

- Completely private, local blockchain
- Instant block generation on command
- No network needed — fully offline
- Perfect for unit/integration tests

Local development & debugging

Testnet: Public Bitcoin Test Network

Experimenting safely with test bitcoins

What is Testnet?

- A parallel Bitcoin network designed for testing and experimentation
- Uses separate "test bitcoins" that have no real monetary value
- Has relaxed consensus rules — faster block times than mainnet
- Faucet services provide free testnet coins (e.g., Bitcoin TestNet Sandbox: testnet.manu.backend.hamburg)

Testnet Commands

```
$ bitcoind --testnet -daemon

$ bitcoin-cli --testnet <command>

$ bitcoin-qt --testnet
```

Regtest: Local Regression Testing Mode

Build and test your own private blockchain

Key Features

- Creates a completely local, private blockchain
- Blocks are generated instantly on demand
- No connection to any external network
- Ideal for automated testing and CI/CD (Continuous Integration Continuous Delivery/Deployment)
- Logs stored in .bitcoin/regtest/debug.log

Regtest Commands

```
$ bitcoind -regtest -daemon

$ bitcoin-cli -regtest generate 200

$ bitcoin-cli -regtest getbalance

$ bitcoin-cli -regtest stop
```

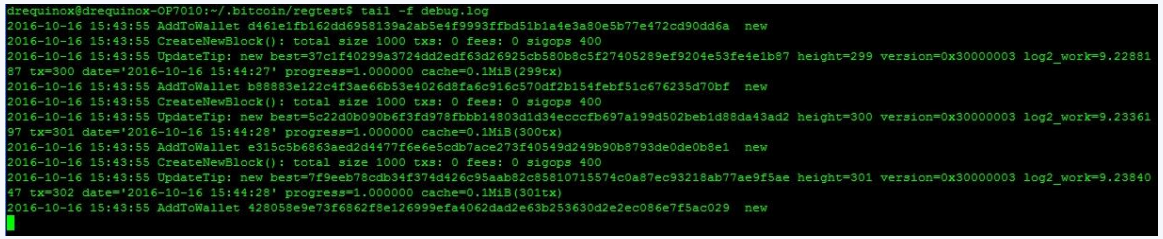


Figure: Bitcoin debug log messages during regtest block generation

Experimenting with bitcoin-cli

The JSON-RPC command-line interface

bitcoin-cli is the primary CLI tool for interacting with the Bitcoin node via JSON-RPC. It provides hundreds of commands to query, control, and manage the local blockchain.

Core Usage

- Query node info: bitcoin-cli getinfo
- List all commands: bitcoin-cli help
- HTTP REST available from Core 0.10.0+ on port 8332
- Useful for scripting, automation, and testing



Figure: bitcoin-cli getinfo output

JSON-RPC: Bitcoin's API Protocol

How clients talk to the Bitcoin daemon



How It Works

- JSON-RPC is a protocol for calling remote functions using JSON-encoded messages
- Bitcoin Core listens for JSON-RPC on TCP port 8332 by default
- All bitcoin-cli commands are JSON-RPC calls sent to bitcoind
- From Core 0.10.0+: also supports HTTP REST on the same port 8332
- External applications can call JSON-RPC directly to build custom Bitcoin tools

Online Bitcoin APIs

Third-party services for Bitcoin development

Bitcoin APIs provide an HTTP REST interface to blockchain data — no need to run a full node.

Blockchain.info

`blockchain.info/api`

Widely used blockchain data & wallet API

BitPay

`bitpay.com/api`

Payment processing and merchant API

Block.io

`block.io`

Multi-currency API including Bitcoin

Bitcore

`bitcore.io`

Full-stack Bitcoin platform by BitPay

BitcoinJS

`bitcoinjs.org`

JavaScript Bitcoin library and API

Bitcoin Programming Libraries

LIBRARIES

Multi-language support for Bitcoin applications

Libbitcoin

C++

`libbitcoin.dyne.org`

Provides powerful command-line utilities, clients, and a comprehensive toolkit for building Bitcoin applications in C++.

Key Features

- Command-line utilities
- Full Bitcoin protocol
- High performance

Pycoin

Python

`github.com/richardkiss/pycoin`

A Python library for Bitcoin key management, transaction creation, and blockchain interaction. Excellent for scripting and automation.

Key Features

- Key management
- Transaction creation
- Script support

Bitcoinj

Java

`bitcoinj.github.io`

A Java Bitcoin library that supports SPV (simplified payment verification) clients. Well-suited for Android and enterprise applications.

Key Features

- SPV wallet support
- Android compatible
- Enterprise-ready

Evaluating Bitcoin APIs: Security Matters

Choosing the right API for your project

> *"Whenever you evaluate an API for usage, in addition to assessing the offered features, also evaluate how secure is the design of the API."*
>
> — *Mastering Blockchain, 2nd Edition*

Features to Assess

- Transaction creation and broadcasting
- Wallet management capabilities
- Address generation support
- Query capabilities (balance, TXs)
- WebSocket/streaming support

Security to Evaluate

- Authentication mechanism (API keys, OAuth)
- HTTPS/TLS enforcement
- Rate limiting and DDoS protection
- Key management — who holds private keys?
- Track record and audits

Key Terms and Definitions

KEY TERMS

Chapter 7 Vocabulary

bitcoind

Bitcoin daemon — background service maintaining the blockchain and providing the JSON-RPC interface.

bitcoin-cli

Command-line interface for interacting with bitcoind via JSON-RPC. Does NOT act on the blockchain independently.

bitcoin-qt

Graphical Bitcoin client with built-in wallet; verifies blocks on disk before displaying the interface.

JSON-RPC

Remote Procedure Call protocol using JSON encoding; Bitcoin's primary API interface, listening on port 8332.

bitcoin.conf

Configuration file storing runtime parameters for Bitcoin Core; located at $HOME/.bitcoin/ on Linux.

Testnet

Public test network using valueless test coins with relaxed mining rules; ideal for integration testing.

Regtest

Regression test mode — a private, local blockchain with instant block generation for development.

Faucet

Service that provides free testnet bitcoins for testing purposes; e.g., Bitcoin TestNet Sandbox.

HTTP REST

Alternative interface available from Bitcoin Core 0.10.0+ on port 8332 for RESTful API access.

Benefits and Limitations

PROS & CONS

Bitcoin client ecosystem — honest assessment

Benefits

- Multiple client types for different use cases (daemon, CLI, GUI)
- Testnet and Regtest enable safe experimentation without real funds
- Rich ecosystem of APIs across multiple languages (Python, Java, C++)
- Open-source — full code transparency and auditability
- HTTP REST interface provides flexibility (from v0.10.0+)
- Active community and extensive documentation

Limitations

- Initial blockchain sync requires significant time and disk space
- bitcoin-cli is completely dependent on bitcoind — cannot function alone
- Selecting an API is difficult — all providers offer similar features
- No universal "best" API recommendation — evaluation required each time
- Security analysis required for every third-party API considered
- Learning curve for JSON-RPC interface and command syntax

Quick Reference: Client & Network Comparison

REFERENCE

Client Types

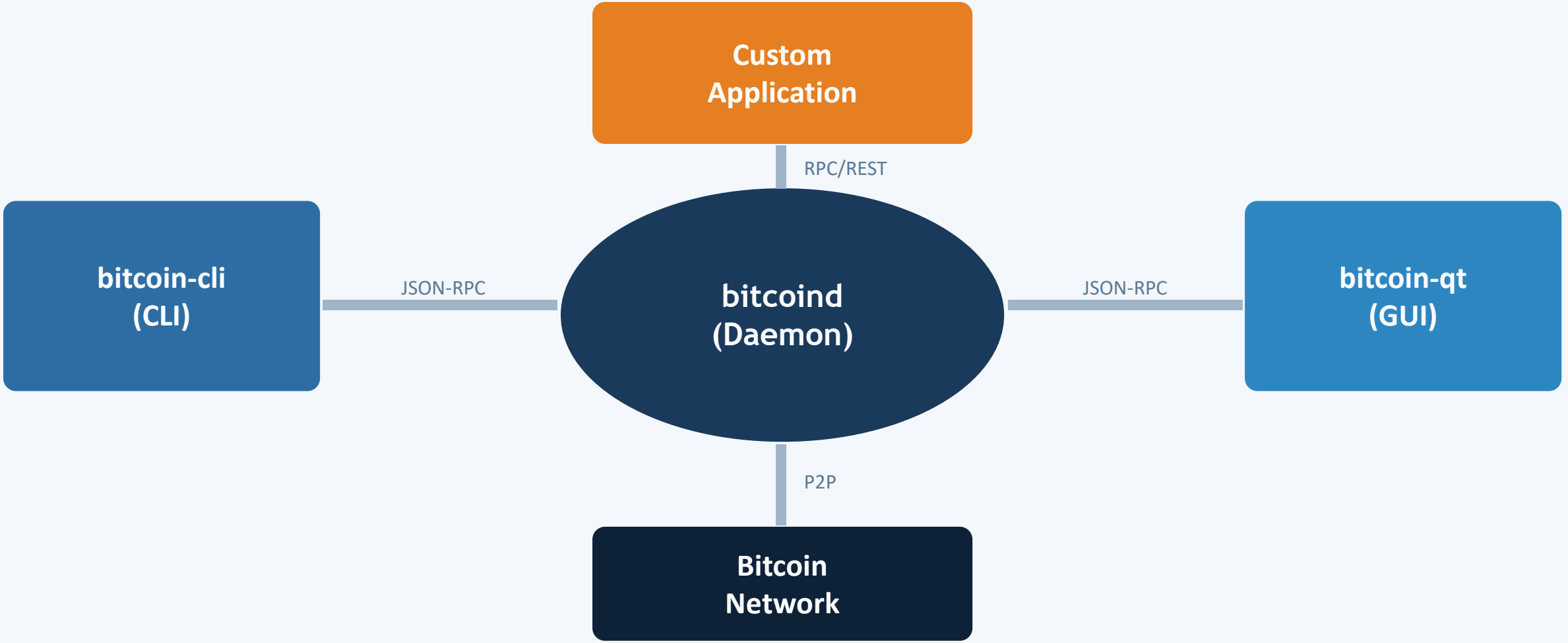
Client	Type	Interface	Depends On	Blockchain Access
bitcoind	Background daemon	JSON-RPC only	Standalone	Direct
bitcoin-cli	CLI tool	Command-line	bitcoind	Via JSON-RPC
bitcoin-qt	GUI application	Graphical (GUI)	Standalone	Direct

Network Modes

Mode	Command Flag	Real Value?	Block Speed	Use Case
Mainnet	(default)	YES — real BTC	~10 minutes	Production
Testnet	--testnet	NO — test coins	Faster	Integration testing
Regtest	-regtest	NO — virtual	Instant	Local development

How the Bitcoin Client Ecosystem Works Together

Architecture overview



bitcoind stores the full blockchain locally. bitcoin-cli and bitcoin-qt interact with bitcoind via JSON-RPC. External apps can use JSON-RPC or HTTP REST.

SECTION 3

Bitcoin Programming
APIs & Libraries

Building applications on top of the Bitcoin protocol

The Bitcoin Development Landscape

DEVELOPMENT

Choosing your tools

Bitcoin programming is a rich field. Developers have multiple options depending on their language preference, use case, and whether they want to run a full node.

Path 1: Full Node + RPC

Run bitcoind locally, interact via bitcoin-cli or JSON-RPC. Maximum control and privacy.

- Best for: node operators, exchanges, full control
- Tools: bitcoind + bitcoin-cli

Path 2: Programming Libraries

Use language-specific libraries without running a full node. Good for transaction building and key management.

- Best for: app developers, wallet builders
- Libraries: Libbitcoin, Pycoin, Bitcoinj

Path 3: Online APIs

Use third-party HTTP APIs. Easiest to start, but requires trust in the provider.

- Best for: rapid prototyping, web apps
- APIs: Blockchain.info, BitPay, Block.io, Bitcore

Hands-On: Try It Yourself

EXERCISE

Practical exercises using Regtest

The safest way to explore Bitcoin is in Regtest mode — no real money, no network needed.

01 **Start Regtest Node**

```
$ bitcoind -regtest -daemon
```

Launch your private local node

02 **Mine Some Blocks**

```
$ bitcoin-cli -regtest generate 101
```

Generate 101 blocks to unlock the mining reward

03 **Check Balance**

```
$ bitcoin-cli -regtest getbalance
```

Verify your balance (should be 50 BTC × mature blocks)

04 **Create an Address**

```
$ bitcoin-cli -regtest getnewaddress
```

Generate a new Bitcoin receiving address

05 **Send a Transaction**

```
$ bitcoin-cli -regtest sendtoaddress <addr> 1.0
```

Send 1 BTC to the new address

06 **Confirm Transaction**

```
$ bitcoin-cli -regtest generate 1
```

Mine 1 more block to confirm the transaction

Key Takeaways

What you should remember from Chapter 7

TAKEAWAYS

1

Three Client Types

Bitcoin Core has three clients: bitcoind (daemon), bitcoin-cli (CLI via RPC), and bitcoin-qt (GUI) — each serving a different purpose.

2

Three Network Modes

Always use the right environment: Mainnet for production, Testnet for integration testing, Regtest for local development.

3

bitcoin.conf Is Central

Centralizes all runtime settings for Bitcoin Core. Located at $HOME/.bitcoin/ on Linux. Command-line flags take precedence.

4

Rich API Ecosystem

Multiple online APIs (Blockchain.info, BitPay, Block.io) and libraries (Libbitcoin, Pycoin, Bitcoinj) support Bitcoin development.

5

Security Is Non-Negotiable

When evaluating third-party Bitcoin APIs, security must be assessed alongside features — not all APIs are equally secure.

6

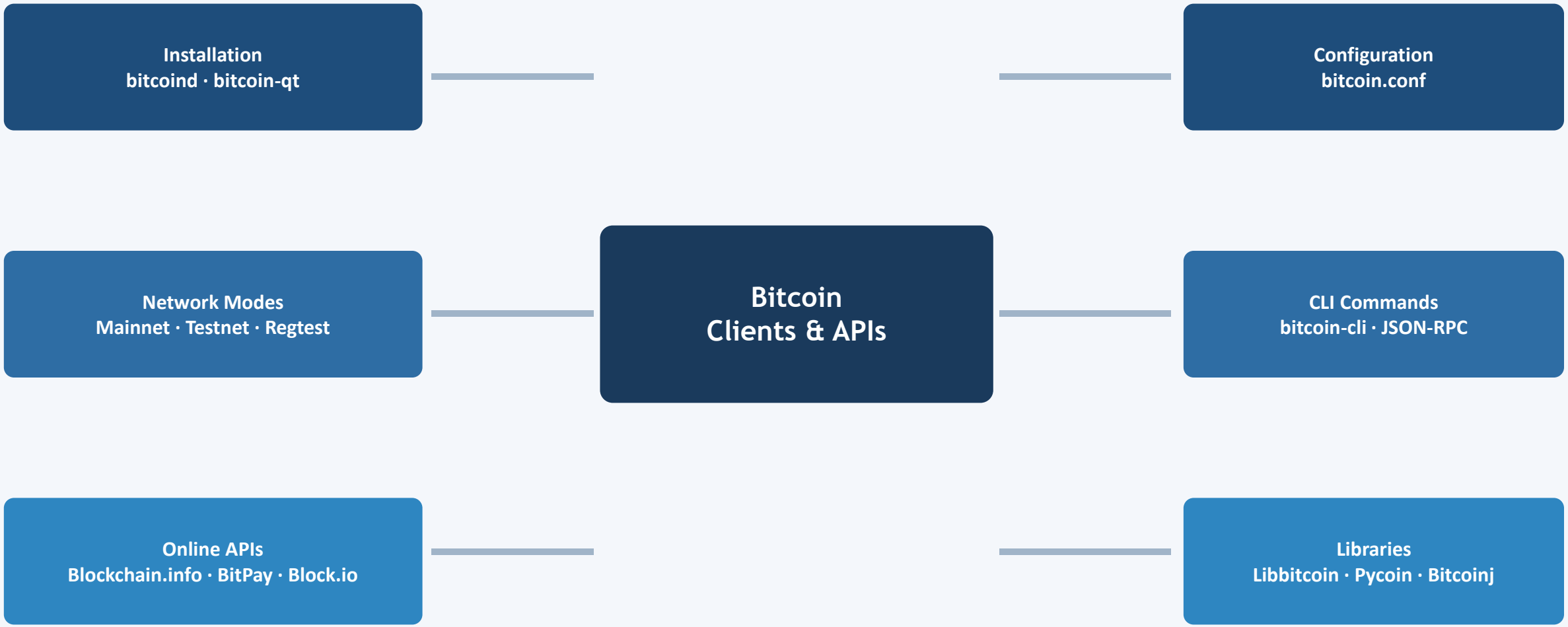
Regtest for Development

Regtest mode is the safest and most efficient way to develop and test Bitcoin applications — instant blocks, no real funds.

Chapter 7 at a Glance

RECAP

Full topic map



Discussion

Questions & Review

Think About It:

Q 1 What is the difference between bitcoind and bitcoin-cli?

Q 2 In which network mode would you run your first Bitcoin test transaction?

Q 3 Name two security factors to evaluate when choosing a Bitcoin API.

Q 4 What does the bitcoin.conf file do, and where is it located on Linux?

Thank You

Chapter 7: Bitcoin Clients and APIs
TC789 – Teknologi Blockchain | FTI UKSW
Teguh Indra Bayu, Ph.D.

Next Chapter
Chapter 8: Alternative Coins
Altcoins, Namecoin, Litecoin, Primecoin, Zcash

Source: Mastering Blockchain, 2nd Edition — I. Bashir, Packt Publishing